

What exactly produced this result?

Daily learning note | 24 September 2026 | Q003

Sidiq review status: Not yet reviewed.

Research question and hypothesis

Can a versioned experiment manifest bind an archived result to declared source bytes and a stated evaluation protocol, while rejecting omissions and unnoticed file changes? Our hypothesis is that all unchanged declared files verify, four required-metadata omissions fail, and a one-byte alteration fails verification.

Q001 and Q002 established information-timing contracts. C001 introduced a checked C++ calculation yesterday. Q003 now adds provenance: a result is easier to audit when we can identify its inputs, logic and protocol. Today follows the oldest ready non-Wednesday backlog item. It does not add a trading signal.

Beginner explanation and terms

Imagine keeping a recipe card beside a cake. The card names the ingredients, the steps and the intended test. A manifest is that card for an experiment. A filename is only a label: its contents can change while the name stays the same. A checksum adds a fingerprint of the actual bytes so a later reader can check for a change.

Manifest: a structured record describing an experiment. **Schema:** the rules for that record's fields. **Provenance:** information about how an output was produced. **Checksum or digest:** a compact value computed from bytes. **SHA-256:** the hash algorithm used here. **Serialization:** turning a structured value into bytes. **Evaluation protocol:** the declared procedure for testing a question. **Synthetic:** invented rather than observed data.

A schema version describes how to interpret a record. An experiment version distinguishes a revised experiment. Changing the research question need not change the schema, but it should produce an identifiable new record.

Mathematics and independent example

Let B_i be the bytes of source file i and d_i the digest stored in the manifest. Compute $h_i = \text{SHA256}(B_i)$. The integrity gate accepts only if metadata is valid and $h_i = d_i$ for every declared file i . Missing files fail. This is an exact comparison of digest strings, without a floating-point tolerance.

SHA-256 returns 256 bits. Each hexadecimal character represents four bits, so the stored digest has $256 / 4 = 64$ hexadecimal characters. Distinct inputs can theoretically share a digest: the output space is finite, whereas possible input files are unbounded. A digest is not a mathematical proof of uniqueness.

Our independent known-answer test uses the exact three bytes `abc`, without a newline. Its expected digest is `ba7816bf8f01cfea414140de5dae2223b00361a396177a9cb410ff61f20015ad`. The expected value is a literal test constant, not calculated by the function being checked.

Data, metrics and protocol declared before implementation

There is no real dataset. The manifest audits Q002's archived eight-row synthetic label example, whose invented information interval is 1-10 January 2026. The Q002 holdout is [5 January, 8 January) in UTC. That period describes the fixture, not actual observed market returns. No model is fitted or tuned.

The declared source set has four files: the archived Q002 JSON result, its generator script, the purging implementation and the dependency lock. Expected audit metrics are four verified files, four rejected missing-metadata cases and detection of one appended byte. Repeat runs must produce identical result JSON. The negative byte-change experiment operates on temporary copies and leaves the originals untouched.

The manifest was recorded today after Q002's results existed. It is retrospective documentation, not evidence that Q002 was preregistered. A timestamp written by an author cannot independently establish prior commitment.

Implementation and input validation

The reusable module is `src/quant_research/validation/manifests.py`. It has separate operations to load JSON, validate metadata, serialize a record and verify its source files. Validating a record alone does not read source files or establish that their bytes match. The experiment explicitly invokes source verification.

Version 1 requires `schema_version`, `experiment_id`, `data_kind`, `question`, `hypothesis`, `information_period`, `evaluation_protocol` and `sources`. Only integer 1 is supported as the schema version; true and 1.0 are rejected. Unknown fields fail rather than being silently ignored. Text must be nonblank. Data kind must be synthetic or real, so mixed data needs separate explicit records for now. Period and protocol text remain human assertions.

Each source requires a unique normalized relative POSIX path and a lowercase 64-character hexadecimal hash. Absolute paths, parent traversal and ambiguous paths fail. Verification resolves paths beneath the supplied root and rejects missing files, directories and symlinks escaping that root. Files must remain unchanged during verification; this mechanism is not a security boundary against a concurrently hostile filesystem.

Python's JSON loader ordinarily accepts repeated object names with the last value winning. We instead reject repeated names, including inside source records, so an ambiguous document cannot silently redefine a hash. Serialization sorts object keys, uses two-space indentation and adds one final newline. Source-list order is preserved. These are project conventions, not a universal canonical JSON format.

Reproduction commands

Run `make manifest-experiment` from the repository root for the audit. Run `make check` for Python tests, formatting, lint, type checking and the existing C++ example. Run `make purging-experiment` to reproduce Q002; its generated bytes must still match the archived checksum. Rebuild this PDF with `make note NOTE_DATE=2026-09-24`.

Observed results and checks

Check	Observed result
Declared source verification	Four of four files matched
Required omission diagnostics	Missing data kind, question, protocol and source hash all rejected
Byte alteration	Appended space detected; original files unchanged
Audit repeatability	Two runs matched each other and stored JSON exactly
Q002 replay	Regenerated output matched archived bytes exactly
Python tests	94 passed, including 45 new cases
Code quality	Ruff passed; 22 files formatted; mypy passed for 14 source files
Existing C++ example	Warning-clean build and expected-output comparison passed
Dependency consistency	No broken requirements found; no new dependency added

The first check attempt found long lines; formatting and one explicit string wrap corrected them before the successful full check. The dependency check disabled an unwritable cache but still passed. No test coverage percentage, market performance, speedup or remote CI success is inferred from these local checks.

Worked interpretation: a harmless-looking edit

Suppose the archived JSON ends with a newline. Appending one space does not change its parsed financial meaning, but it does change its bytes. The pinned digest no longer matches and the verifier raises an error. This is intentional: the contract is exact file identity, not semantic equivalence of JSON documents.

A failure is a prompt to investigate. It could mean an intentional formatting change, a revised experiment, a stale file or an unintended edit. Do not regenerate the expected hash automatically just to pass the test. For an intentional revision, retain the earlier version in Git history and record a new experiment ID with new evidence. Verify historical manifests against the corresponding historical checkout.

Limitations and interpretation

A checksum proves neither truth nor authorship. If a party changes both a file and its expected digest, this comparison alone cannot reveal the replacement. There is no signature, trusted timestamp or access-control system here. Declaring data real or synthetic is still an author assertion; the validator does not inspect markets.

The four-file manifest is a bounded provenance record, not a complete dependency graph or a fully captured execution environment. A lock file does not record the entire operating system, hardware or compiler. Matching sources does not establish licensing, point-in-time validity, absence of bias or successful reproduction. Reproduction additionally requires executing the procedure and comparing outputs.

No empirical alpha is claimed. Q005a's fold bug remains open; use UTC-normalized inputs for Q001.

Primary sources and decisions

Official sources consulted on 24 September 2026. The schema and fixture choices are project decisions.

1. Python hashlib documentation describes SHA-256 and binary `file_digest`. We hash exact binary file contents and store the hexadecimal digest; no dependency is needed beyond the standard library.
<https://docs.python.org/3/library/hashlib.html>
2. Python json documentation describes repeated-name behavior, `object_pairs_hook` and sorted-key output. We use these controls to reject duplicates and produce stable project serialization.
<https://docs.python.org/3/library/json.html>
3. W3C PROV overview describes provenance in terms of entities, activities and agents. This motivates keeping artifacts and the procedure together. Our small schema is not a standards-compliant PROV implementation. <https://www.w3.org/TR/prov-overview/>

What to explain in an interview

Start with the failure mode: a stored result and its filename can survive even after the input file changes. Describe the schema validation gate separately from the source-integrity gate, then show the one-byte negative test. Explain why the expected digest is trusted only as far as the stored record is trusted. Finish by stating that correct provenance supports reproducibility but does not establish predictive value or a sound experiment.

Next eligible work

Q004 is next: specify an investable universe and compare data-access options, including delistings, revisions and licence constraints. Q005a remains a required fix before general ingestion. C002 is ready for the next Wednesday slot, 30 September. No personal review approval is required to continue these authorised tasks.

Check your understanding

1. Why is a filename insufficient to identify the source used in a result?
2. Why does SHA-256 produce 64 hexadecimal characters, and does that prove two files are identical?
3. Why are metadata validation and source verification separate operations?
4. Should an appended space fail the file checksum even if JSON still parses identically?
5. If every checksum matches, what remains unproven about the research?

Suggested answers

1. The contents can change without renaming the file. A pinned digest records a fingerprint of the bytes that the author intended to identify, allowing later changes to be detected against that reference.
2. There are 256 bits and four bits per hexadecimal digit, giving 64 digits. Matching hashes are strong practical evidence of matching bytes under this use, but finite hash spaces permit theoretical collisions.
3. One validates the record's shape and declared values; the other reads the actual files. A well-formed record can still point to missing or changed files, so the audit must perform both operations.
4. Yes. This contract identifies bytes, including whitespace and line endings. Semantic JSON comparison would be a different contract and would require an explicitly defined normalization procedure.
5. Authorship, licensing, complete environment capture, data truth, leakage freedom, successful execution and empirical validity remain unproven. The record helps investigate those questions; it does not answer them.